i think you have missed few steps -

1. we did setup ruleset for release/* branches
2. we added some labels under issues section

Can you revisit all the steps from scratch and provide end-to-end step by step guide for implementation along with the files

Yes — you were right. For the full cascade setup on a fresh repository, the workflow files are only one part; you also need the **release branch ruleset** and the **repository labels** that support the manual/conflict flow. GitHub supports all three pieces natively: workflow YAMLs in `.github/workflows/`, branch protection through branch rulesets or protection rules, and labels from the repository's Issues/PR labels page.[1] [2] [3]

What to set up

For each new repository, set up these parts in this order:

- release branches,
- branch ruleset for `release/*`,
- repository labels,
- repository secret for the PAT,
- workflow files,
- and one controlled end-to-end test. Rulesets can target branch name patterns, and labels apply across both issues and pull requests in the same repository.[4] [2] [5]

Prerequisites

Keep these ready before starting:

- admin or maintainer access to the repository,
- the ordered release branches you want to cascade across,
- the final `cascade-next-pr.yml` and `cascade-conflict-check.yml` files,
- and a PAT to store as a repository Actions secret. Repository secrets are created under **Settings** > **Secrets and variables** > **Actions**, and rulesets can target matching branch patterns like `release/*`.[5] [4]

Step 1: Create the release branches

Create all required release branches in the order your cascade expects, for example from the earliest downstream branch to the latest downstream branch. Your cascade logic depends on those release branches already existing, so do not rely on the workflow to invent the branch chain for you.

You can create them from Git locally:

```
git checkout main
git pull
git checkout -b release/1.0
git push -u origin release/1.0

git checkout main
git checkout -b release/1.1
git push -u origin release/1.1

git checkout main
git checkout -b release/1.2
git push -u origin release/1.2
```

If your team already has a release branch naming convention, keep it identical because the workflow behavior depends on predictable branch names. GitHub branch protection and rulesets can then target them using a pattern instead of one-by-one setup.[6] [4]

Step 2: Add the release branch ruleset

Go to the repository on GitHub and open:
Settings > **Rules** > **Rulesets**. GitHub rulesets are the current mechanism for controlling how users interact with specific branches and tags, and they can target patterns such as `release/*`. [4] [1]

Create a **branch ruleset** for:

```
release/*
```

At minimum, configure it so release branches are controlled through PRs and not by casual direct pushes. Depending on what you had previously set, typical controls include:

- require a pull request before merging,
- require approvals if your process expects them,
- require status checks if your workflows expose checks,
- restrict force pushes,
- restrict deletions,
- and optionally restrict who can bypass. GitHub documents these as available controls within branch protection/rulesets for protected branches.[7] [6] [4]

If your earlier setup allowed workflow-driven merges through a bot or PAT user, make sure the ruleset does **not** accidentally block that account from completing the cascade. Rulesets support bypass allowances for selected roles, teams, or apps, which is important when automation must still merge protected branches.[1] [4]

Step 3: Add repository labels

Open the repository and go to:
Issues or Pull requests > **Labels** > **New label.** GitHub labels are shared across issues and pull requests, so once created they can be used by your workflow or manual triage flow on PRs as well.[2] [8]

Create the exact labels your cascade process expects. Since I do not want to invent label names you may have used earlier, use the same names and colors from your previous repository setup. Common examples in cascade/conflict flows are labels for conflict, manual action needed, cascade PR, or auto-merge blocked, but you should keep them exactly aligned with your earlier implementation if the workflow references label names literally. GitHub lets you create, edit, and reuse labels from the labels page.[8] [2]

A practical way to verify this is to open both workflow files and search for any `label`, `labels`, or GitHub API calls that add labels. If the workflow uses literal label strings, those exact labels must already exist in the repository or the labeling step may fail or become inconsistent.

Step 4: Add the PAT secret

Go to:
Settings > **Secrets and variables** > **Actions** > **New repository secret.** GitHub repository secrets are the standard secure way to provide tokens to Actions workflows.[5]

Create the secret:

```
CASCADE_GITHUB_TOKEN
```

Paste the PAT value there. That PAT should have the repository permissions required for the tasks your workflow performs, such as creating PRs, updating branches, adding labels, or merging pull requests.[9] [5]

Step 5: Create the workflow folder

In the repository, create this folder if it does not already exist:

```
.github/workflows/
```

GitHub only recognizes workflow definitions that are committed into `.github/workflows/`.[3] [10]

Step 6: Add the workflow files

Create these two files:

```
.github/workflows/cascade-next-pr.yml
.github/workflows/cascade-conflict-check.yml
```

Paste in the final workflow contents you previously validated. GitHub workflows are defined by YAML files in that folder, and they become active after you commit and push them to the repository.[10] [11]

Step 7: Verify secret usage inside the workflow

Open `cascade-next-pr.yml` and `cascade-conflict-check.yml` and confirm they reference the repository secret correctly, for example with:

```
${{ secrets.CASCADE_GITHUB_TOKEN }}
```

GitHub Actions secrets are consumed in workflow syntax through the `secrets.<NAME>` context, so the secret name in the repository must match the workflow exactly.[11] [5]

Step 8: Commit and push the files

From the repo root:

```
git add .github/workflows/cascade-next-pr.yml .github/workflows/cascade-conflict-che
git commit -m "Add cascade workflows"
git push
```

After the push, GitHub should detect and register the workflows for that repository.[3] [10]

Step 9: Verify workflows appear in Actions

Open the repository's **Actions** tab and confirm both workflows are listed. GitHub shows installed workflows there once the YAML files are present in the default branch or an accessible branch that GitHub can detect for workflow runs.[12] [10]

Step 10: Cross-check the implementation before test

Before running a real cascade, verify these items carefully:

- release branches already exist and match the expected naming pattern,
- the ruleset is applied to `release/*`,[4]
- the exact labels from the old repo exist in the new repo,[2]
- `CASCADE_GITHUB_TOKEN` exists as an Actions secret,[5]
- both workflow files are committed under `.github/workflows/`,[3]

- and the workflows reference the exact label names and secret names you created.

Step 11: Run the first end-to-end test

Use one small, controlled PR:

1. Create a normal PR into the first release branch.
2. Merge it manually according to your release process.
3. Watch the cascade workflow run in **Actions**.
4. Confirm it creates the PR into the next release branch.
5. Confirm the right labels are applied if your workflow is supposed to apply them.
6. Confirm clean downstream PRs auto-merge only where your earlier setup intended that behavior.
7. Confirm conflict cases stop and are marked for manual resolution.

GitHub Actions run history and logs are available in the Actions tab, while labels on PRs can be checked directly in the pull request sidebar.[13] [10] [2]

Expected repository structure

The minimum file structure in the repo should look like this:

```
<repo-root>/
└── .github/
    └── workflows/
        ├── cascade-next-pr.yml
        └── cascade-conflict-check.yml
```

That is the required location for GitHub workflow YAML files.[10] [3]

Implementation checklist

Use this exact order on every new repository:

- Create all `release/*` branches first.
- Add a branch ruleset targeting `release/*`.[4]
- Configure the protections your earlier process used, especially PR-based updates and restricted direct pushes.[6] [7]
- Create the repository labels used by the cascade/conflict flow.[2]
- Add `CASCADE_GITHUB_TOKEN` as a repository Actions secret.[5]
- Add `cascade-next-pr.yml` and `cascade-conflict-check.yml` under `.github/workflows/`.[3]
- Push the files and verify both workflows appear in **Actions**.[10]
- Run one controlled cascade test.

About the files

I can describe the file paths and where each one goes, but I should not fabricate the workflow contents because you said these are the workflows you set up previously, and the exact YAML, branch pattern logic, labels, and merge conditions need to match that earlier implementation. The safest next step is for you to share either the old workflow YAMLs or the old repository name, and I can then turn this into a precise copy-paste setup guide with the exact files, exact labels, and exact ruleset values.

What I need from you next

Send me **any one** of these and I'll produce the final exact end-to-end guide:

- the old `cascade-next-pr.yml`,
- the old `cascade-conflict-check.yml`,
- the list of label names you used,
- or screenshots/text of the `release/*` ruleset settings.

Then I can give you a **complete, exact, repo-from-scratch implementation guide** with no missing steps.

⁂

1. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets
2. https://docs.github.com/en/issues/using-labels-and-milestones-to-track-work/managing-labels
3. https://docs.github.com/en/actions/get-started/quickstart
4. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets
5. https://docs.github.com/actions/security-guides/using-secrets-in-github-actions
6. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/managing-a-branch-protection-rule
7. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches
8. https://docs.github.com/en/enterprise-server@3.15/issues/using-labels-and-milestones-to-track-work/managing-labels
9. https://github.com/marketplace/actions/create-github-secret-action
10. https://docs.github.com/en/actions/tutorials/create-an-example-workflow
11. https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions
12. https://www.youtube.com/watch?v=ylEy4eLdhFs
13. https://github.com/orgs/community/discussions/53473
14. https://github.com/orgs/community/discussions/16007

15. https://docs.github.com/actions/deployment/targeting-different-environments/using-environments-for-deployment
16. https://www.youtube.com/watch?v=hQZ2Bm1GhTE
17. https://github.com/sbellone/release-workflow-example
18. https://github.nih.gov/about/features/protected-branches
19. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/creating-rulesets-for-a-repository
20. https://github.com/github/docs/blob/main/content/issues/using-labels-and-milestones-to-track-work/managing-labels.md